



카테고리 없음

[Denoising Diffusion Probabilistic Models] 논문 리뷰

yuha933 2026. 3. 25. 14:04

1. Introduction

연구 배경

- 최근 Deep Generative Models은 다양한 데이터 도메인에서 고품질 샘플 생성 성능을 보여왔으며, 이미지 및 오디오 생성에서 매우 높은 품질을 달성했다.
 - GAN (Generative Adversarial Networks)
 - Autoregressive Models
 - Flow-Based Models
 - VAEs (Variational Autoencoders)
- 또한, GAN과 경쟁 가능한 이미지 생성 성능을 보이는 모델들도 등장하기 시작했다.
 - EBM (Energy-Based Models)
 - Score Matching 기반 Model

기존 연구(Diffusion)의 한계

- Diffusion 모델이 존재하기는 했으나, 고품질 샘플 생성 능력이 입증되지 않았다.
- 기존 확률 모델들과 비교했을 때, log-likelihood 성능의 경쟁력이 부족했다.
- 모델이 많은 비트를 사람이 인지 못하는 디테일 표현에 낭비했다.

Diffusion Probabilistic Model 제안

1. Forward process

- 원래 이미지 x_0 에 아주 조금씩 Gaussian noise를 계속 추가하는 과정
- 모델이 학습하는 과정이 아니라, 사람이 정의하는 과정을 말한다.

2. Reverse process

- noise를 없애 원래 이미지 x_0 로 다시 되돌리는 과정
- 모델이 학습하는 과정으로, 한 번에 복원하는 것이 아니라 한 step씩 조금씩 복원한다.

※ noise가 Gaussian이면, reverse도 Gaussian으로 설정 가능하다.

[Forward process]

x_0 (원본 이미지)
-> x_1 (조금 흐림)
-> x_2 (더 흐림)
-> ...
-> x_T (noise)

[Reverse process]

x_T (Noise)
-> x_{T-1}
-> x_{T-2}
-> ...
-> x_0 (원본 이미지)

본 논문의 기여

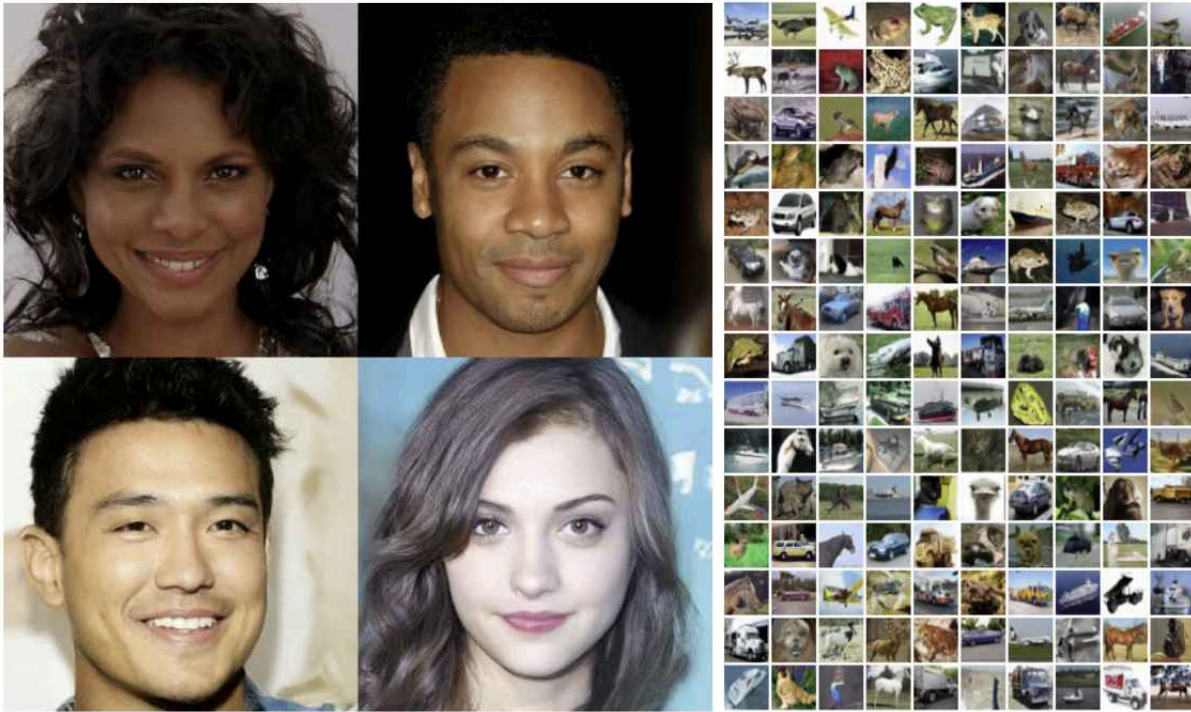


Figure 1

- Diffusion 모델이 실제로 고품질 이미지 생성이 가능함을 최초로 입증하였으며, 일부 경우에는 기존 모델보다 우수함을 보인다.
- Training은 Denoising Score Matching 과정과 같고, Sampling은 Langevin Dynamics와 같아 Diffusion을 새로운 모델로 보기보다 기존 score-based generative model 과 같은 계열로 볼 수 있는 이론적 연결성을 확보했다.
- Gaussian 기반 transition을 사용함으로써, 복잡한 구조 없이 학습이 가능함을 보인다.
- 특정 순서로 생성하는 것이 아니라 bit ordering 자체를 일반화하였다.

2. Background

Reverse Process : 데이터를 생성하는 모델을 정의하자 !!

$$p_{\theta}(\mathbf{x}_{0:T}) := p(\mathbf{x}_T) \prod_{t=1}^T p_{\theta}(\mathbf{x}_{t-1}|\mathbf{x}_t), \quad p_{\theta}(\mathbf{x}_{t-1}|\mathbf{x}_t) := \mathcal{N}(\mathbf{x}_{t-1}; \boldsymbol{\mu}_{\theta}(\mathbf{x}_t, t), \boldsymbol{\Sigma}_{\theta}(\mathbf{x}_t, t))$$

- noise에서 시작해서 이미지를 생성하는 과정
 - 시작 : $\mathbf{x}_T \sim \mathcal{N}(0, \mathbf{I})$
 - $\mathbf{x}_T \rightarrow \mathbf{x}_{T-1} \rightarrow \mathbf{x}_{T-2} \rightarrow \dots \rightarrow \mathbf{x}_0$ 로 가면서 점점 덜 noisy한 상태로 만든다.
- 한 번에 이미지를 생성하는 것이 아니라, 조금씩 복원하는 형태의 모델이다.\
- 다만, $\mathbf{x}_T$ 만 보고 $\mathbf{x}_{T-1}$ 의 정답 분포를 알 수 없다.

Forward Process : 반대로 가는 과정을 우리가 만들자 !!

$$q(\mathbf{x}_{1:T}|\mathbf{x}_0) := \prod_{t=1}^T q(\mathbf{x}_t|\mathbf{x}_{t-1}), \quad q(\mathbf{x}_t|\mathbf{x}_{t-1}) := \mathcal{N}(\mathbf{x}_t; \sqrt{1 - \beta_t}\mathbf{x}_{t-1}, \beta_t \mathbf{I})$$

- 이미지에 noise를 조금 추가하는 과정
 - $x_0 \rightarrow x_1 \rightarrow x_2 \rightarrow \dots \rightarrow x_T$ 로 가면서 결과 완전 noise가 된다.
- 이 과정은 학습을 하지 않고, reverse를 forward 기반으로 학습할 수 있게 된다.

Training Objective (ELBO) : 진짜 likelihood 대신 계산 가능한 목표를 최적화하자 !!

- $\log p_\theta(x_0)$ 가 필요하지만, 직접 계산은 어렵다.
- 그래서 ELBO를 도입한다.

$$\mathbb{E}[-\log p_\theta(\mathbf{x}_0)] \leq \mathbb{E}_q \left[-\log \frac{p_\theta(\mathbf{x}_{0:T})}{q(\mathbf{x}_{1:T}|\mathbf{x}_0)} \right] = \mathbb{E}_q \left[-\log p(\mathbf{x}_T) - \sum_{t \geq 1} \log \frac{p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)}{q(\mathbf{x}_t|\mathbf{x}_{t-1})} \right] =: L$$

- 다만, ELBO는 식이 매우 복잡하고, 직관적이지 않아 계산이 어렵고 그로 인해 학습이 불가능하다.

ELBO 분해 : ELBO를 시간 step별로 쪼개자 !!

$$\mathbb{E}_q \left[\underbrace{D_{\text{KL}}(q(\mathbf{x}_T|\mathbf{x}_0) \parallel p(\mathbf{x}_T))}_{L_T} + \sum_{t \geq 1} \underbrace{D_{\text{KL}}(q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) \parallel p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t))}_{L_{t-1}} - \underbrace{\log p_\theta(\mathbf{x}_0|\mathbf{x}_1)}_{L_0} \right]$$

- ELBO를 풀어쓰면, 각 step마다 모델이 진짜 분포를 잘 맞추고 있는지에 대한 비교 형태로 변한다.

$$q(x_{t-1}|x_t, x_0) \quad vs \quad p_\theta(x_{t-1}|x_t)$$

- 다만, 우리는 $q(x_{t-1} | x_t, x_0)$ 를 알아야 KL 계산이 가능하다.

Forward process 기반으로 이 posterior를 계산해보자 !!

$$q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) = \mathcal{N}(\mathbf{x}_{t-1}; \tilde{\boldsymbol{\mu}}_t(\mathbf{x}_t, \mathbf{x}_0), \tilde{\beta}_t \mathbf{I}),$$

where $\tilde{\boldsymbol{\mu}}_t(\mathbf{x}_t, \mathbf{x}_0) := \frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t}{1 - \bar{\alpha}_t} \mathbf{x}_0 + \frac{\sqrt{\bar{\alpha}_t}(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t} \mathbf{x}_t$ and $\tilde{\beta}_t := \frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t} \beta_t$

- posterior가 Gaussian으로 close-form이 존재한다.
- KL 계산, Loss 계산, gradient 계산이 모두 가능하다.

3. Diffusion models and denoising autoencoders

3.1 Forward process and L_T

- forward variance β_t 는 그냥 고정값으로 쓴다.
- Forward process에서 q 는 학습할 게 없기 때문에, 결론적으로 L_T 는 상수가 된다. 따라서, 학습에 영향이 없으므로 무시해도 된다.

$$L_T = D_{KL}(q(x_T|x_0)||p(x_T))$$

3.2 Reverse process and $L_{\{1:T-1\}}$

- 목표 : 모델 $p_\theta(x_{t-1} | x_t)$ 가 진짜 posterior $q(x_{t-1} | x_t, x_0)$ 를 맞추게 한다.

Gaussian 구조 활용

- KL 최소화하려면 평균을 맞추면 되므로, 모델은 posterior mean을 맞추도록 하면 된다.

$$L_{t-1} = \mathbb{E}_q \left[\frac{1}{2\sigma_t^2} \|\tilde{\mu}_t(\mathbf{x}_t, \mathbf{x}_0) - \mu_\theta(\mathbf{x}_t, t)\|^2 \right] + C$$

문제 전환

- 문제 : 정답에 x_0 가 필요한데, 모델은 x_t 만 본다.
- 해결 : x_t 를 다시 표현한다.

$$q(\mathbf{x}_t | \mathbf{x}_0) = \mathcal{N}(\mathbf{x}_t; \sqrt{\alpha_t} \mathbf{x}_0, (1 - \alpha_t) \mathbf{I})$$

- 결과적으로, loss가 μ 를 맞추기보다 ϵ 를 맞추는 것으로 변환되었고, 이미지 복원 문제에서 노이즈 예측 문제로 전화되었다.

최종 parameterization

$$\mu_\theta(\mathbf{x}_t, t) = \tilde{\mu}_t \left(\mathbf{x}_t, \frac{1}{\sqrt{\alpha_t}} (\mathbf{x}_t - \sqrt{1 - \alpha_t} \epsilon_\theta(\mathbf{x}_t)) \right) = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1 - \alpha_t}} \epsilon_\theta(\mathbf{x}_t, t) \right)$$

- 수식 의미 : 이 이미지에 섞인 노이즈는 무엇인가 (denoising 문제)

최종 Loss 형태

$$\mathbb{E}_{\mathbf{x}_0, \epsilon} \left[\frac{\beta_t^2}{2\sigma_t^2 \alpha_t (1 - \alpha_t)} \left\| \epsilon - \epsilon_\theta(\sqrt{\alpha_t} \mathbf{x}_0 + \sqrt{1 - \alpha_t} \epsilon, t) \right\|^2 \right]$$

- 단순 MSE로, 실제 noise와 모델이 예측한 noise를 비교한 값이다.

Algorithm 1 (Training)

Algorithm 1 Training

```

1: repeat
2:    $\mathbf{x}_0 \sim q(\mathbf{x}_0)$ 
3:    $t \sim \text{Uniform}(\{1, \dots, T\})$ 
4:    $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ 
5:   Take gradient descent step on
        $\nabla_{\theta} \|\epsilon - \epsilon_{\theta}(\sqrt{\alpha_t}\mathbf{x}_0 + \sqrt{1 - \alpha_t}\epsilon, t)\|^2$ 
6: until converged

```

Algorithm 2 (Sampling)

Algorithm 2 Sampling

```

1:  $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ 
2: for  $t = T, \dots, 1$  do
3:    $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$  if  $t > 1$ , else  $\mathbf{z} = \mathbf{0}$ 
4:    $\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left( \mathbf{x}_t - \frac{1 - \alpha_t}{\sqrt{1 - \alpha_t}} \epsilon_{\theta}(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}$ 
5: end for
6: return  $\mathbf{x}_0$ 

```

3.3 Data scaling, reverse process decoder, and L_0

Data scaling

- 원본 이미지 : 0 ~ 255
- 모델 입력 : [-1, 1]
- Gaussian 기반 모델에 맞추기 위해 이미지를 연속 공간으로 변환한다.

Decoder의 필요성

- 문제 : 모델은 x_0 를 연속 값으로 출력하고, 실제 데이터는 이산 픽셀값이라 직접 비교가 불가능하다.
- 해결 : 마지막 단계에 discrete decoder를 추가한다.
 - decoder는 Gaussian 분포를 바로 픽셀값으로 쓰지 않고, 각 픽셀 구간에 들어갈 확률로 변환하는 역할을 한다.
- 결과적으로, 실제 이미지 데이터에 대해 올바른 log-likelihood 계산이 가능하다.

L_0의 의미

- 마지막 복원 단계의 loss로, x_1 에서 x_0 으로의 복원 정확도를 의미한다.

3.4 Simplified training objective

Section 3.2의 결과

$$\mathbb{E}_{\mathbf{x}_0, \epsilon} \left[\frac{\beta_t^2}{2\sigma_t^2 \alpha_t (1 - \bar{\alpha}_t)} \left\| \epsilon - \epsilon_\theta(\sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, t) \right\|^2 \right]$$

- timestep마다 가중치가 존재하기 때문에, 가중치가 복잡하고 학습 불안정의 문제가 있다.

simple loss : 가중치를 제거하자 !!

$$L_{\text{simple}}(\theta) := \mathbb{E}_{t, \mathbf{x}_0, \epsilon} \left[\left\| \epsilon - \epsilon_\theta(\sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, t) \right\|^2 \right]$$

- t값이 작으면 noise가 적어 쉬운 문제로 인식하고, 크면 noise가 많아 어려운 문제로 인식한다.
- 따라서, 어려운 문제에 더 집중할 있어, 샘플 품질 향상을 돕는다.

4. Experiments

실험 설정

- T = 1000
- β_t : 0.0001 → 0.02 (선형 증가)
- 모델 : U-Net + self-attention
- positional embedding 사용

4.1 Sample quality

실험 내용

- CIFAR-10 기준
- metrics
 - IS (Inception Score)
 - FID
 - NLL

결과

Model	IS	FID	NLL Test (Train)
Conditional			
EBM [11]	8.30	37.9	
JEM [17]	8.76	38.4	
BigGAN [3]	9.22	14.73	
StyleGAN2 + ADA (v1) [29]	10.06	2.67	
Unconditional			
Diffusion (original) [53]			≤ 5.40
Gated PixelCNN [59]	4.60	65.93	3.03 (2.90)
Sparse Transformer [7]			2.80
PixelIQN [43]	5.29	49.46	
EBM [11]	6.78	38.2	
NCSNv2 [56]		31.75	
NCSN [55]	8.87 ± 0.12	25.32	
SNGAN [39]	8.22 ± 0.05	21.7	
SNGAN-DDLS [4]	9.09 ± 0.10	15.42	
StyleGAN2 + ADA (v1) [29]	9.74 ± 0.05	3.26	
Ours (L , fixed isotropic Σ)	7.67 ± 0.13	13.51	≤ 3.70 (3.69)
Ours (L_{simple})	9.46 ± 0.11	3.17	≤ 3.75 (3.72)

- FID = 3.17로, 기존 모델들보다 우수하며, conditional 모델보다도 좋다.
- Diffusion은 likelihood 최적화보다 샘플 품질 측면에서 강력하다.

샘플 결과



Figure 3: LSUN Church samples. FID=7.89



Figure 4: LSUN Bedroom samples. FID=4.90

- 매우 자연스러운 구조로, texture 및 lighting이 안정적이다.

4.2 Reverse process parameterization and training objective ablation

실험 내용

- μ 예측 (baseline)
- ε 예측 (본 논문 제안)
- variance 학습 vs. 고정

결과

Objective	IS	FID
$\tilde{\mu}$ prediction (baseline)		
L , learned diagonal Σ	7.28 ± 0.10	23.69
L , fixed isotropic Σ	8.06 ± 0.09	13.22
$\ \tilde{\mu} - \tilde{\mu}_\theta\ ^2$	—	—
ϵ prediction (ours)		
L , learned diagonal Σ	—	—
L , fixed isotropic Σ	7.67 ± 0.13	13.51
$\ \tilde{\epsilon} - \epsilon_\theta\ ^2 (L_{\text{simple}})$	9.46 ± 0.11	3.17

- μ 예측 : variational bound에서만 잘 작동하고, simple objective에서는 성능 나쁘다.
- ϵ 예측 : simple objective에서 가장 좋다.
- variance 학습 : unstable하고, 성능이 낮다.
- 결과적으로, 노이즈 추정 문제로 바꾸는 것이 핵심이다.

4.3 Progressive coding

Progressive lossy compression

실험 내용

- rate-distortion 분석 수행

결과

- train/test gap이 매우 작으며, overfitting이 없다.
- 다만, likelihood 성능이 SOTA는 아니다.
- 결과적으로, Diffusion은 좋은 Generative 모델이지만 likelihood 모델로 최고는 아니다.

그래프 결과

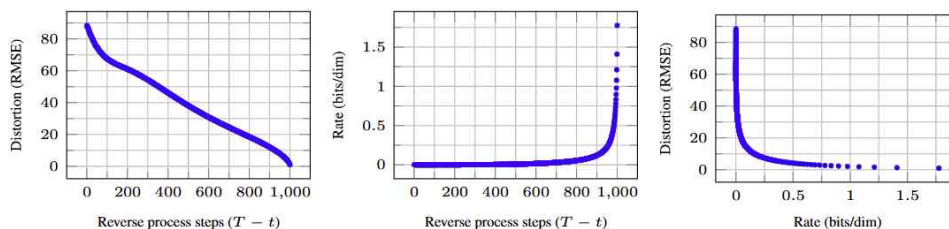


Figure 5: Unconditional CIFAR10 test set rate-distortion vs. time. Distortion is measured in root mean squared error on a $[0, 255]$ scale. See Table 4 for details.

- 초반에는 distortion이 급격히 감소한다.

- 대부분의 bit는 미세 디테일에 사용된다.

Progressive generation

실험 내용

- 생성 과정 중간 상태 관찰

그래프 결과



Figure 6: Unconditional CIFAR10 progressive generation ($\hat{x}_0$ over time, from left to right). Extended samples and sample quality metrics over time in the appendix (Figs. 10 and 14).



Figure 7: When conditioned on the same latent, CelebA-HQ 256×256 samples share high-level attributes. Bottom-right quadrants are x_t , and other quadrants are samples from $p_\theta(x_0|x_t)$.

- 초기에는 큰 구조만 생성하고, 후반에 가서 디테일을 생성한다.
- diffusion은 coarse에서 fine 생성 구조를 유지한다.

Connection to autoregressive decoding

Diffusion = autoregressive의 일반화

- Diffusion objective를 재해석하면 autoregressive 형태로 변환 가능하다.

4.4 Interpolation

실험 내용

- latent space에서 interpolation 수행
- reverse process로 복원

결과



Figure 8: Interpolations of CelebA-HQ 256x256 images with 500 timesteps of diffusion.

- diffusion latent space는 의미 있는 구조를 가진다.

5. Related Work

flows / VAEs

- flows/VAEs : latent 데이터 정보 유지
- Diffusion : 완전한 noise로 만들어버림

Score Matching / Langevin Dynamics

- Diffusion은 새로운 모델이 아니라 score-based 모델을 variational inference로 표현한 것이다.
 - training : denoising score matching
 - sampling : annealed Langevin dynamics

Markov chain

- 본 연구는 Markov chain 기반 생성 모델은 기존에도 있었지만 Diffusion은 가장 tractable하고 단순한 구조이다.

Energy-Based Model (EBM)

- Diffusion은 EBM까지 확장 가능한 프레임워크이다.

Rate-Distortion / Compression

- diffusion은 단순 생성 모델이 아니라 compression 모델로 해석 가능하다.

Progressive decoding

- Diffusion 생성 과정은 한 번에 복원이 아니라, 점진적인 복원으로 이루어진다.

Autoregressive Models

- Diffusion은 autoregressive 모델의 일반화된 형태이다.

6. Conclusion

의의

- 본 논문은 diffusion 모델을 통해 고품질 생성이 가능함을 입증하고, 이를 score matching·Langevin dynamics·variational inference와 연결하여 생성 모델의 통합적 프레임워크를 제시한 연구이다.

Future work

- 다른 데이터(audio, etc.)로 확장 가능하다.
- 다른 Generative Model 구성 요소로 활용될 수 있다.